

Section 3: System Design Fundamentals

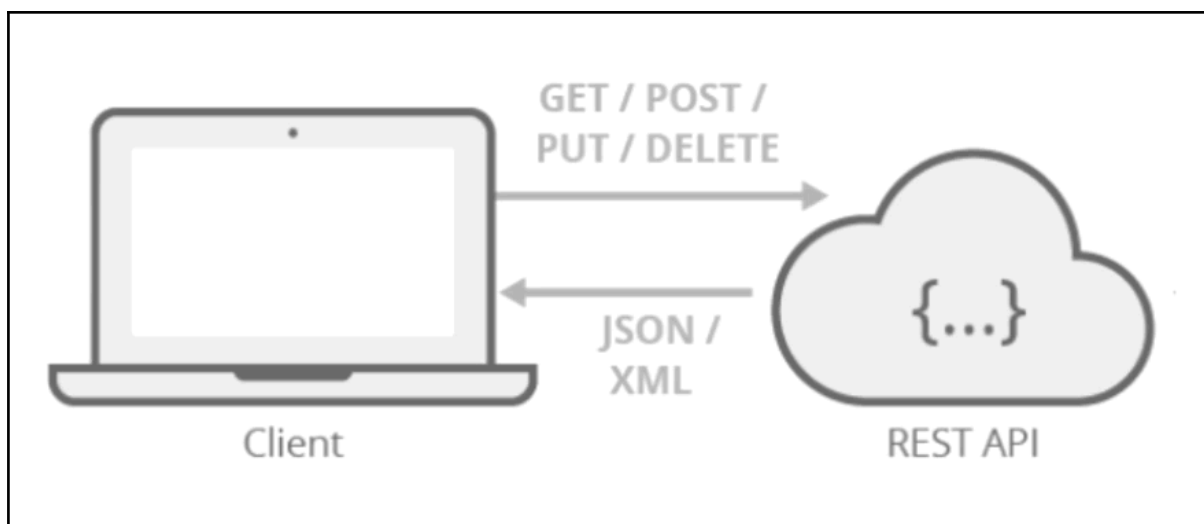
REST & RESTfulness - API Design Principles

In this session, we will dive deep into REST architecture, its core constraints and best practices for designing efficient, scalable and secure RESTful APIs. Understanding REST is crucial for system design interviews, as it's the foundation of modern web services and APIs.

By the end of this session, you will have a clear grasp on how REST works, how to structure an API and have a look at the real-world applications like Twitter and GitHub APIs.

What is REST?

REST (Representational State Transfer) is an architectural style used for designing networked applications. Instead of relying on complex protocols, REST leverages standard web protocols like HTTP to enable communication between clients and servers. The main idea behind REST is that it operates on stateless communication, meaning each request from a client contains all necessary information and the server does not store any session data between requests. This makes REST scalable, reliable and very easy to implement.



The concept of REST was introduced by Mr. Roy Fielding in his 2000 doctoral dissertation. His work laid the foundation of how modern web services, including APIs and other services used by big companies like Twitter, GitHub and Google are built today.

Understanding REST is fundamental to designing efficient and scalable APIs, which is why it plays a major role in system design interviews.

Why REST?

REST is widely used in modern web applications and system design due to the following reasons:

- **Simplicity:** REST is built on standard HTTP methods like GET, POST, PUT and DELETE, making it easy to understand and implement.
- **Scalability:** REST follows a stateless architecture, it scales efficiently allowing multiple servers to handle requests without maintaining any session data.
- **Interoperability:** REST APIs are platform-independent, meaning they can be consumed by clients running on different devices, different OS and implemented in different programming languages. Whether it's a web application, mobile application or even IoT devices, REST can be used everywhere.
- **Efficiency:** It is the major reason why REST is widely adopted. By leveraging caching, REST can introduce lower latencies. It reduces unnecessary requests. It improves response times and it reduces the overall server loads. Since, REST is stateless, it avoids overhead associated with maintaining client sessions, making it highly efficient for distributed systems.

So because of these benefits, REST has become the backbone of modern API design and it powers everything from social media platforms to cloud services.

REST Constraints (Core Principles)

To truly understand REST, we need to explore its core principles, also known as REST constraints. These constraints define how RESTful systems should be designed to ensure scalability, reliability and simplicity.

- **Client-Server Architecture:** This constraint separates concerns between the clients and the server, that means a request and a response. This separation allows for independent development and scalability, meaning the frontend and backend can evolve separately.
- **Statelessness:** Each request from a client must contain all the necessary information for the server to process it. The server does not store session data, which ultimately improves scalability and simplifies system management. This is why REST is ideal for distributed systems and cloud-based applications.
- **Cacheability:** REST allows responses to be cached to improve the performance and reduce the unnecessary requests to the server. This is particularly important for high traffic APIs because caching helps seeding up the responses and reduces the server load.

- **Layered System:** REST APIs can have multiple layers such as security, load balancing and proxy servers. Without affecting how clients interact with the API, these layers can function independently. This layered approach enhances the overall flexibility, security and stability.
- **Uniform Interface:** This is the most defining characteristic of REST. It ensures that all the resources are accessed in a consistent manner using standard HTTP methods like GET, POST, PUT and DELETE. This uniformity simplifies the development as APIs follow a predictable structure.

So by following these constraints, RESTful APIs achieve scalability, flexibility and efficiency, and thus making them the preferred choice for modern web services.

RESTful API Design Principles

RESTful API design principles are some of the popular best practices widely used for designing well-structured and efficient RESTful APIs. The below are the principles:

- **Resource-Based Approach:** In REST everything is treated as a resource, such as users, orders, products. If you have these entities, these objects in your application, these are treated as a resource. These resources are identified using URIs, which are Uniform Resource Identifiers. So instead of thinking in terms of actions like GET user or UPDATE order, we focus on resources. Examples:
 - GET /users/{id} to retrieve a user
 - POST /orders to create a new order
- **Proper HTTP Methods Usage:** REST API leverages standard HTTP verbs to perform operations on resources.
 - GET to retrieve data
 - POST to create new resources
 - PUT to update existing resources
 - PATCH for partial updates
 - DELETE to remove resources

Using the correct method makes the API much more intuitive, predictable, and aligned with RESTful principles.

- **Stateless Interactions:** Each request must contain all the information, and the server does not store any client session data between requests. This simplifies scaling because any request can be handled by any available server. Authentication and session-related things, if you want to have in your REST APIs, can be typically done using tokens like JWT tokens instead of session-based storage.

- **Consistency in URL Structure:** Maintaining consistency in URL structures is very crucial for a well-designed API. Resource URLs should be clear, hierarchical, and predictable. So there are some principles like:
 - Use plural nouns for collections: /users, /orders
 - Avoid including actions in URLs:
 - ❌ /users/{id}/activate
 - ✅ /users/{id} with PATCH
 - Implement versioning for backward compatibility: /v1/users

So by following these principles, we can ensure that RESTful APIs are intuitive, scalable and easy to use. A well structured API follows clear and logical endpoint patterns to ensure consistency and usability. By structuring resources and endpoints this way, we ensure the API is clean, predictable, and easy to use.

JSON vs. XML in REST APIs

When we are designing a RESTful API, choosing the right data format for request and response payload is also very crucial. The two commonly used formats are JSON and XML.

XML	vs.	JSON
<pre> 1 <?xml version="1.0" encoding="UTF-8"?> 2 <endereco> 3 <cep>31270901</cep> 4 <city>Belo Horizonte</city> 5 <neighborhood>Pampulha</neighborhood> 6 <service>correios</service> 7 <state>MG</state> 8 <street>Av. Presidente Antônio Carlos, 6627</street> 9 </endereco> </pre>		<pre> 1 { 2 "endereco": { 3 "cep": "31270901", 4 "city": "Belo Horizonte", 5 "neighborhood": "Pampulha", 6 "service": "correios", 7 "state": "MG", 8 "street": "Av. Presidente Antônio Carlos, 6627" 9 } 10 } </pre>

JSON nowadays has become a standard for the REST APIs because of several advantages.

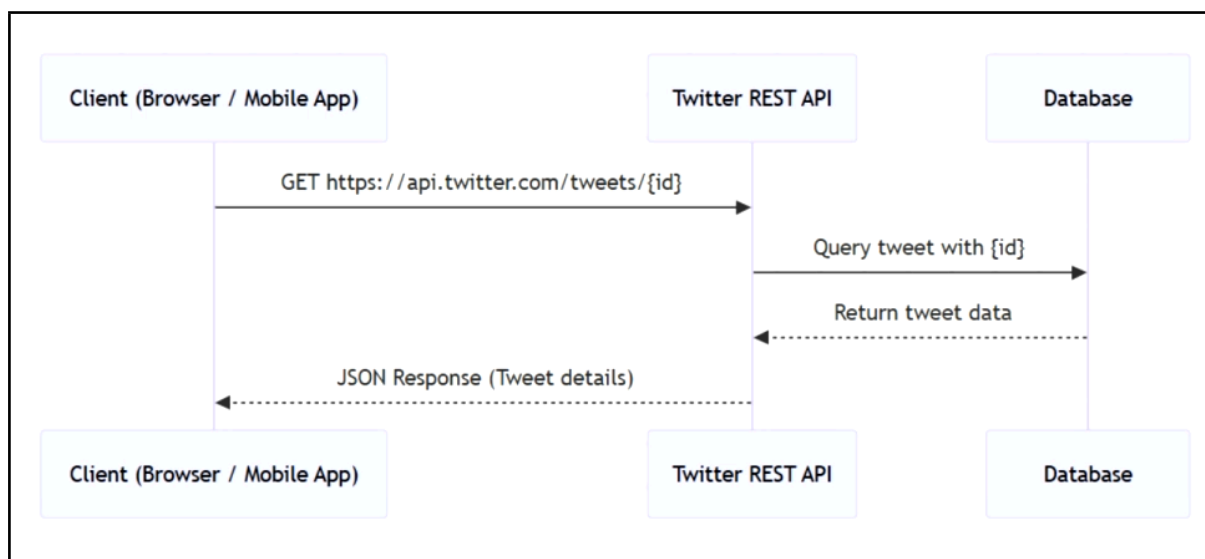
- ➔ **Lightweight:** It uses fewer characters and a simpler structure.
- ➔ **Faster:** It is natively supported in JavaScript and can be quickly parsed by the modern web and mobile applications.
- ➔ **Readable:** JSON's key-value format is easy for both developers and machines to understand.

But, there are still some scenarios where XML is still used in specific use cases. Use cases like if you have legacy systems. Many older enterprise applications and SOAP-based services still rely on XML. There could be some data validation needs. XML supports schemas, XSDs, which do strict data validations, making it useful for scenarios where formal data structures are required.

Most modern REST APIs use JSON due to its simplicity and efficiency. XML is still relevant for legacy integrations and scenarios requiring strict schema validations. There are some APIs that support both JSON and XML support, which allows the client to specify their preferred format using an accept header in the actual calls. That is called the content negotiation of the actual format.

Real-World REST API Examples

We have covered the principles of RESTful API, let's look at a real world example of REST APIs used by major platforms like Twitter and GitHub. These APIs follow RESTful principles to provide a structured scalable and efficient interaction with their services.



If you look at the twitter API example, we have two examples here, one to fetch the tweets using GET and another one to post a tweet using POST. Then GitHub has some repos to get the repo details and to create issues in GitHub.

Best Practices & Common Pitfalls

Some of the best practices that will help you ensure that APIs are well designed, scalable and secure. And we will also look at some pitfalls.

- **Use Proper Status Codes:** HTTP status codes provide clear feedback on the outcome of API requests. Using the right status code improves error handling and debugging for both developers and clients.
- **API Versioning:** APIs evolve over time, so versioning prevents breaking changes. A common approach to versioning is in the URLs, but there could be alternative approaches where you can actually put version information in the header.
- **Implement Authentication and Authorization:** Secure APIs with authentication mechanisms like OAuth2 or JWT tokens, and it will be always better. Also try to always use HTTPS for protecting your sensitive data.
- **Pagination:** If you have a large set of data, always try to implement pagination. Large data sets should be paginated to improve performance. You can use query string parameters like page and limit to actually specify the paging parameters and get paginated data. This reduces server load and improves the API responsiveness.
- **Avoid Using Verbs in URLs (Common Pitfall):** We should avoid using verbs in the URL. So a user is a resource. Everytime you want to create a user, you should just do a POST on the user's resource rather than having a verb in your URL that says something like /createUser.

By following these best practices, we can ensure that APIs are robust, scalable and secure.

Summary & Key Takeaways

- REST is a stateless, scalable, and widely adopted API design style.
- Proper endpoint structure, HTTP methods, and response formats matter.
- Security, versioning, and best practices improve API quality.
- What's next:
 - Real-Time Communication Protocols

Interview Questions - REST

Basic Questions

- What is REST, and how does it differ from SOAP?
- What are the six constraints of REST architecture?
- What is the difference between a REST API and a RESTful API?
- What is a resource in REST, and how is it represented?
- What are endpoints in a REST API?

HTTP Methods & Status Codes

- Explain the difference between GET, POST, PUT, PATCH, and DELETE.
- When would you use PUT vs. PATCH?
- What are the commonly used HTTP status codes in REST APIs?
- What does the 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, and 500 Internal Server Error mean?

RESTful API Design & Best Practices

- What are the best practices for designing RESTful APIs?
- How do you design a RESTful API for a blogging platform (e.g., users, posts, comments)?
- What is HATEOAS (Hypermedia as the Engine of Application State) in REST?
- How do you handle authentication and authorization in REST APIs?
- What are API rate limiting and throttling, and why are they important?
- How do you ensure backward compatibility when updating a REST API?

Advanced & Real-World Questions

- How does caching work in RESTful APIs?
- What is the difference between JSON and XML in REST?
- How do you implement pagination in REST APIs?
- How does versioning work in REST APIs?
- Explain the differences between REST, GraphQL, and gRPC.
- How would you improve the performance of a REST API?
- Can a REST API be stateful? Why or why not?
- How does REST handle security vulnerabilities like SQL injection and CSRF?